

Mobile Edge Deployment and Resource Management for Maritime Wireless Networks

Chaoyue Zhang, *Member, IEEE*, Bin Lin , *Senior Member, IEEE*, Ziru Chen , *Member, IEEE*,
Lin X. Cai , *Senior Member, IEEE*, and Jianli Duan 

Abstract—Mobile Edge Computing (MEC) has been envisioned as one of the key technologies for supplying computation and storage resources in Internet of Vessels (IoV) networks. Due to its flexible deployment, low cost and agile maneuverability, Unmanned Surface Vehicle (USV) has emerged as a promising solution, to provide communication and computation services for maritime users. In this paper, we study mobile edge deployment and resource management for MEC-assisted maritime wireless networks where USVs with diverse computation resources are deployed to provide edge computing services that complement the cloud-based services. To this end, we formulate an optimization problem to minimize the expected response time by jointly optimizing the deployment of mobile USVs and computation offloading decisions. To solve the mixed-integer nonlinear program problem, we propose a Dual-Layer Reinforcement Learning (DLRL) framework to attain a near-optimal solution. Specifically, a Deep Deterministic Policy Gradient (DDPG) algorithm is designed to obtain the best USV deployment in the outer layer learning, and a Q-learning algorithm is designed to determine the best computation offloading decisions in the inner layer learning. Numerical results demonstrate that the proposed solution outperforms some literature algorithms by effectively handling both continuous and discrete variables.

Index Terms—Internet of Vessels (IoV), maritime wireless networks, unmanned surface vehicle (USV), mobile edge deployment and resource management, deep reinforcement learning.

I. INTRODUCTION

INTERNET of Vessels (IoV) becomes an indispensable part of maritime wireless communication networks, which enables many smart maritime applications, such as environment monitoring [1], intelligent navigation and voice/video streams, etc. The rapid growth in requirements for maritime applications leads to an increased demand for high bandwidth, low latency, and enhanced computation capabilities. Both academia and the

maritime industry conduct research on the deployment of maritime wireless networks to meet the increasing demand from IoV users. Maritime wireless communication networks are typically classified into shore-based communication networks and offshore-based communication networks. Shore-based communication networks can utilize 3G/4G/5G communication technologies to provide broadband data transmissions, typically covering a radius of up to thirty kilometers from the shore. For offshore-based communication networks, communication services can be provisioned through High Frequency (HF) and Very High Frequency (VHF) channels, and maritime satellites. Typical offshore communication systems, such as PACTOR system, Automatic Identification System (AIS) and VHF Data Exchange System (VDES), operate in the HF/VHF band [2]. These systems usually offer low data rate services over a large communication coverage, e.g., several hundreds of kilometers. In contrast, maritime satellites can offer high data rate services worldwide from anywhere [3]. However, the high cost of vessel-borne equipment and communication expenses pose significant challenges for small vessels. In general, existing maritime wireless networks may not meet the application requirements for communication and computation resources in IoV.

Mobile Edge Computing (MEC) provides proximate access to users with computation resources through the deployment of edge servers [4], [5]. Compared with cloud computing, MEC consumes less transmission time and energy due to a shorter transmission distance [6], [7], [8]. In terrestrial networks, MEC has received extensive attention [9], [10], [11], [12]. However, research on MEC-assisted maritime wireless networks is lagging behind that in the terrestrial networks. Applying MEC in maritime wireless networks presents new research challenges compared to those in terrestrial networks. First, despite the vast expanse of the ocean surface, finding a stable area for deploying an edge server is a challenging task. Second, maintenance and management of edge servers in maritime environments, such as applying security patches, troubleshooting, and hardware repairs, may incur high operating costs. Third, it is challenging to ensure low-latency transmission from maritime users to the edge due to the hostile communication environment of the ocean. Owing to these complex factors, transmission efficiency is often low in maritime environments.

In this context, Unmanned Surface Vehicles (USVs) can serve as portable edge servers in close proximity to maritime users, leveraging the advantages of high mobility and ever-reducing cost [13], [14], [15]. Given the limited computation resource of an individual USV, how to deploy multiple USVs and design offloading strategies to improve the computation resource

Received 23 April 2024; revised 24 September 2024; accepted 17 December 2024. Date of publication 27 December 2024; date of current version 20 May 2025. The work of Bin Lin was supported in part by the National Natural Science Foundation of China under Grant 62371085 and Grant 51939001 and in part by the Fundamental Research Funds for the Central Universities under Grant 3132023514. The review of this article was coordinated by Dr. Phone Lin. (Corresponding author: Bin Lin.)

Chaoyue Zhang and Bin Lin are with Information Science and Technology College, Dalian Maritime University, Dalian 116026, China (e-mail: zcy_11335577@163.com; binlin@dlmu.edu.cn).

Ziru Chen and Lin X. Cai are with the Department of Electrical and Computer Engineering, Illinois Institute of Technology, Chicago, IL 60616 USA (e-mail: zchen71@hawk.iit.edu; lincal@iit.edu).

Jianli Duan is with the School of Science, Qingdao University of Technology, Qingdao 266520, China (e-mail: duanjianli@qut.edu.cn).

Digital Object Identifier 10.1109/TVT.2024.3521393

utilization, to reduce the computation overload, and to reduce the response time of processing tasks is a critical issue in MEC-assisted maritime wireless networks. It is worth noting that the USV deployment and computation offloading decisions are mutually coupled. Different deployments of USVs may lead to different offloading decisions to achieve the minimum response time, and vice versa. Therefore, USV deployment and computation offloading decisions need to be jointly optimized to minimize the response time. To the best of our knowledge, there have been no prior studies on jointly optimizing multi-USV deployment and computation offloading based on the demands of maritime users in an MEC-assisted maritime wireless network.

In this paper, we formulate a joint optimization problem for USV deployment and computation offloading decisions in MEC-assisted maritime wireless networks. Our objective is to minimize the expected response time, which is comprised of the uplink transmission time, the computation time at the edge server or cloud, and the downlink transmission time. The formulated problem is a Mixed-Integer Non-Linear Program (MINLP) problem with both continuous and discrete variables, i.e., the deployment locations of USVs in a continuous space and the binary offloading decisions in a discrete space. It is worth noting that the deployment locations and the computation offloading decisions are tightly coupled, i.e., the best offloading decisions are dependent on the locations of USVs, and vice versa. In addition, the hybrid variables in both continuous and discrete space pose significant challenges in the optimization framework, as MINLP is well known to be NP-hard. To the best of our knowledge, existing solutions typically involve discretizing continuous actions [16], [17], or employing continuous functions to approximate discrete actions [18], [19], or integrating different tools to handle different types of variables, e.g., to utilize learning techniques for continuous variables and matching theory for discrete variables [20], [21]. These solutions may result in reduced accuracy and/or increased computational complexity. To this end, we first reformulate the optimization problem as a Markov Decision Process (MDP), and propose a novel Dual-Layer Reinforcement Learning (DLRL) framework that integrates Deep Deterministic Policy Gradient (DDPG) and Q-learning to handle the complex hybrid action spaces. Specifically, a DDPG algorithm is designed to optimize the locations of USVs, and a Q-learning algorithm is designed to determine the best computation offloading decisions. Finally, extensive simulation results verify that the proposed DLRL algorithm can effectively reduce the expected response time, in comparison with the benchmark algorithms in the literature. The main contributions of this work can be summarized as follows.

- We propose a framework for mobile edge deployment and resource management in MEC-assisted maritime wireless networks, where computation tasks of each vessel can be offloaded to an edge server or cloud center. A joint USV deployment and computation offloading problem is formulated to minimize the expected response time, subject to the constraint of edge computation resources.
- To solve the formulated optimization problem, we reformulate it as an MDP problem, and propose a DLRL framework that integrates DDPG and Q-learning to handle the discrete and continuous actions to facilitate collaborative training.
- In the DLRL algorithm, the outer layer DDPG is designed to optimize the locations of USVs in continuous space,

based on the locations and computation tasks of vessels from historical experiences; and the inner layer Q-learning is designed to determine the best computation offloading decisions in discrete space based on the locations of USVs. Further analysis is conducted on the complexity of the proposed learning algorithm.

The remainder of this paper is organized as follows. The related works are presented in Section II. An MEC-assisted maritime wireless network model and problem formulation are introduced in Section III. A DLRL algorithm is proposed to solve the optimal USV deployment and computation offloading problem in Section IV. Numerical results are presented to evaluate the performance of the proposed DLRL algorithm in Section V, followed by concluding remarks in Section VI.

II. RELATED WORKS

Computation offloading in MEC networks has been extensively studied in the literature [12], [22], [23]. Generally, the computation tasks can be processed at mobile users, or be offloaded to the edge server or the the cloud server. In [12], a DDQN algorithm is applied to jointly optimize the computation offloading decisions, and spectrum and computation resource allocation in an MEC network with a single edge server, aiming to minimize the energy consumption of the system. Computation offloading in an MEC network with multiple edge servers is studied in [22], [23]. In [22], convex optimization methods are used to optimize the computation offloading, the computation resource allocation, and transmission power control in an MEC network, with the objective of minimizing the overall energy consumption. In [23], an end-to-end deep reinforcement learning algorithm is proposed to optimize the offloading decisions and computational frequency allocation in an MEC network with multiple users and multiple edge servers, aiming to maximize the number of tasks completed on time while minimizing the energy consumption.

The deployment of edge servers can be categorized into two types, i.e., 1) fixed deployment, where edge servers are strategically placed within the network infrastructure, with the location unchanged after deployment to ensure stable and reliable edge services; and 2) on-demand deployment, where edge servers are deployed based on network dynamics, and the configuration of the edge servers is adjustable in response to the changing network conditions and demands.

Some studies investigate deploying edge servers among a set of fixed candidate locations. In [24], leveraging the advantages of the bipartite graph and heuristic algorithm, a two-step optimization method is proposed to address the edge server deployment and task scheduling, aiming to maximize the profits of the service provider. In [25], a deployment algorithm based on k-means clustering and particle swarm optimization is proposed to solve the cloudlet deployment problem, with the objective of minimizing the average service delay. In these works, a single objective optimization is formulated. There are some works that formulate the edge server deployment problem as a multi-objective optimization problem [9], [26], [27]. In [26], the objective is to balance the workload and minimize the access delay. Genetic algorithm and local search algorithm are designed to find the best solution. The work in [27] studies how to strike a trade-off among the coverage rate, the waiting time, and the

workload balance. The Niche Pareto Genetic Algorithm II (NPGA-II) is adopted to find the feasible solutions for edge server deployment. In these aforementioned works, heuristic algorithms are proposed to find the deployment solutions, and the deployment is not adaptive to the dynamic demands of the network.

Recent advancement on Unmanned Aerial Vehicles (UAVs) offers a viable and flexible solution for MEC where UAVs equipped with edge servers can provide on-demand services to users in the proximity and the locations of UAVs can dynamically adapt to the user demands. The trajectories of UAVs are usually modeled as continuous variables, and efficient methods and algorithms based on machine learning and convex optimization are proposed to dynamically optimize decision-making processes. The deployment of a single UAV equipped with an edge server is studied in [28], [29]. In [28], an alternative algorithm based on convex optimization and Lyapunov-based approach is proposed to optimize the resource allocation and UAV trajectory in a UAV-assisted MEC system, with the objective of minimizing the average energy consumption. In [29], an alternative algorithm based on block coordinate descent and successive convex approximation techniques is proposed to solve the computation offloading decisions, bits allocation and UAV trajectory optimization problem in a UAV-enhanced MEC network, with the objective of minimizing the total energy consumption. A few research works study multi-UAV deployment and computation offloading in the multi-UAV-enabled MEC network [18], [30], [31], [32]. Generally, joint optimization of multiple UAV trajectories and offloading decisions in a wireless network is a complex optimization problem which is usually modeled as an MINLP problem [33]. Deep reinforcement learning offers an efficient solution to tackle the challenging problem. In [18], a Q-learning algorithm and transport theory are proposed to optimize the multi-UAV deployment and user association, with the objective of minimizing the transmit power consumption. The work in [30] develops a multi-agent DDPG algorithm to jointly optimize the vehicle association, resource allocation and multi-UAV trajectory in an MEC- and UAV-assisted vehicular network. The algorithms used in [18], [30] utilize the approximation method to handle the complex optimization problem, resulting in performance loss. The work in [31] applies a many-to-one matching to optimize the subchannel allocation, and leverages a DDPG algorithm to train the multi-UAV trajectory in a multi-UAV collaborative caching network, aiming to minimize the total content retrieving delay. In [32], an algorithm based on the transport theory and particle swarm intelligence is proposed to find the optimal user association and multi-UAV deployment, with the objective of minimizing the average task delay. The algorithms used in [31], [32] utilize iteration optimization to separately solve the continuous and discrete variables to attain a local optimal solution with high computational complexity. In addition, the existing works mainly consider a small scale MEC network with no more than four UAVs. The complexity of these algorithms usually increases exponentially with the number of UAVs, and thus they are not suitable for a medium or a large scale deployment of MEC with a large number of mobile edge servers.

Similar to UAVs, USVs equipped with edge servers can provide mobile edge services to users in a maritime wireless

network. Joint resource management and USV deployment are studied in [34], [35], and successive convex approximation, semidefinite programming relaxation, interior-point and block coordinate descent methods are proposed to solve the joint optimization problem. Numerical results show that a USV deployed in the maritime wireless network can greatly improve the network throughput. Multi-USV deployment is studied in [36], where bisection search is used to find the optimal placement of USVs along with the data uploading duration and signal powers to achieve the minimum energy consumption. In this work, the number of USVs is no more than six.

The aforementioned literature works focus on either computation task offloading or edge server deployment in an MEC network. Basically, offloading decisions should be adaptive to the dynamic demands of users, which is not generally coupled with network deployment in traditional MEC with fixed deployment of edge servers. Thanks to the mobility of USVs, it is possible and favorable to deploy mobile edge servers according to the dynamic network demands. Therefore, a joint optimization of the edge server deployment and computation offloading beckons for further research. In addition, while most existing works only consider a limited number of USVs/UAVs, a scalable solution is required for a maritime wireless network because a large number of USVs are required to cover a wide area in marine. To the best of our knowledge, our work is the first to jointly optimize multi-USV deployment and computation offloading based on the demands of maritime users in an MEC-assisted maritime wireless network.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Network Model

In this paper, we consider an MEC-assisted maritime wireless network consisting of a cloud center, a coastal Base Station (BS), a group of USVs denoted as $\mathcal{U} = \{1, \dots, U\}$ and a group of vessels denoted as $\mathcal{M} = \{1, \dots, M\}$, as illustrated in Fig. 1. Each vessel is associated with a USV via a wireless link, and USVs can communicate with a coastal BS, which is connected to the cloud center via a wired link. The BS serves as the network controller that collects the network information for mobile edge deployment and resource management. Each USV can be equipped with computation and storage units to provision edge computing services for vessels in the proximity. Due to the limited computation resources in each individual USV, multiple USVs can be deployed in one area to form a cluster of USVs so that the vessel associated with the USVs in the cluster can share the aggregated computing resources. Without loss of generality, USVs are grouped into N clusters, and each USV cluster may contain one¹ or multiple USVs. Denote the set of USVs in USV cluster $n \in \mathcal{N} = \{1, \dots, N\}$ as $\mathcal{Z}_n = \{1, \dots, Z_n\}$, $\forall z_n \in \mathcal{Z}_n$, where $\sum_{n=1}^N Z_n = U$ ($U \geq N$) USVs. In a 2D Cartesian coordinate system, the location of i is denoted as $\mathbf{L}_i = [x_i, y_i]$, where i can be a vessel $i \in \mathcal{M}$, or a USV cluster $i \in \mathcal{N}$. Assume that the task arrival rate of vessel m follows Poisson distribution with mean λ_m . Let I_m denote the number of computation tasks of vessel m in a time slot. The main notations are listed in Table I.

¹This is the special case that each USV will be individually deployed without clustering.

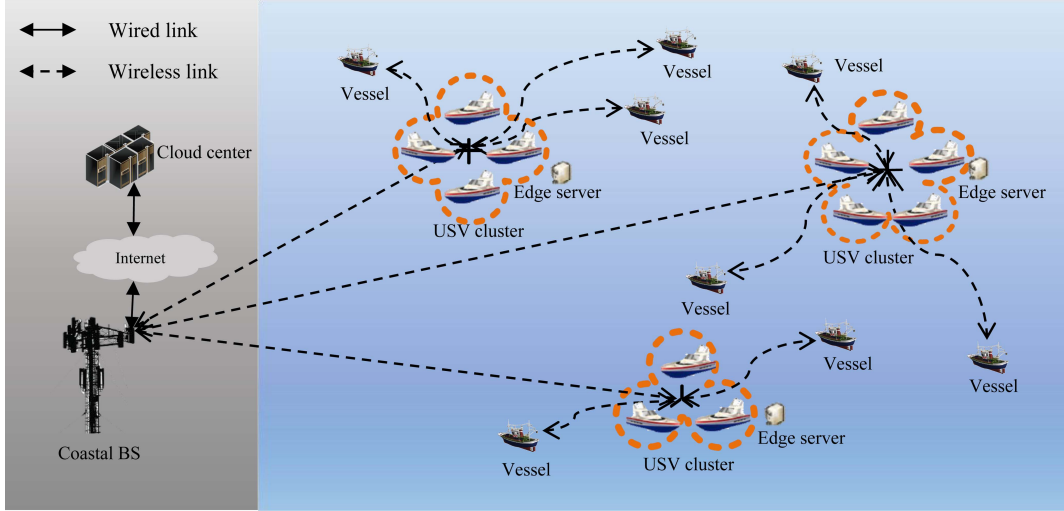


Fig. 1. An MEC-assisted maritime wireless network with multiple USVs.

 TABLE I
MAIN NOTATIONS AND DESCRIPTIONS

Notation	Description
$\mathcal{U}$	A set of USVs
$\mathcal{M}$	A set of vessels
$\mathcal{N}$	A set of USV clusters
$\mathcal{Z}_n$	A set of USVs in USV cluster n
$\alpha_{m,n}$	Offloading decision variable. 1 means vessel m offloads task to USV cluster $n \in \mathcal{N}$ or the cloud center $n = \{0\}$; and 0 vice versa.
λ_m	The task arrival rate of vessel m
I_m	The number of computation tasks of vessel m
c_m	The required CPU cycles for processing the computation task of vessel m
b_m^{in}	The average input data volume of the computation task of vessel m
b_m^{out}	The average data volume of the computation task computation result of vessel m
$\mathbf{L}_i$	The location of i , where $i \in \mathcal{M} \cup \mathcal{N}$
P_i	The transmit power of i , where $i \in \mathcal{M} \cup \mathcal{N}$
H_i	The antenna height of i , where $i \in \mathcal{M} \cup \mathcal{N}$
B	Channel bandwidth
$h_{i,j}$	The channel gain between i and j , where $i, j \in \mathcal{M} \cup \mathcal{N}$
$R_{i,j}$	The transmission rate from i to j , where $i, j \in \mathcal{M} \cup \mathcal{N}$
C_n	The computation resource of n , where $n \in \mathcal{N} \cup \{0\}$

Considering the impacts of sea surface reflection and antenna height, a two-ray signal transmission model is adopted for maritime channels [3]. The transmission model assumes that the maritime channel mainly consists of a direct path and a reflection path, the channel gain between vessel m and the USV cluster n can be expressed as

$$h_{m,n} = \left(\frac{l}{4\pi d_{m,n}} \right)^2 \left[2 \sin \left(\frac{2\pi H_m H_n}{l d_{m,n}} \right) \right]^2, \quad \forall m \in \mathcal{M}, \forall n \in \mathcal{N}, \quad (1)$$

where l is the carrier wavelength, H_m is the antenna height of vessel m , H_n is antenna height of n -th USV cluster header, $d_{m,n} = \sqrt{(x_m - x_n)^2 + (y_m - y_n)^2}$ is the communication link distance. The uplink transmission rate $R_{m,n}$ from

vessel m to USV cluster, and the downlink transmission rate $R_{n,m}$ from USV cluster n to vessel m are

$$R_{m,n} = B \log_2 \left(1 + \frac{P_m h_{m,n}}{\sigma^2} \right), \quad \forall m \in \mathcal{M}, \forall n \in \mathcal{N}, \quad (2)$$

$$R_{n,m} = B \log_2 \left(1 + \frac{P_n h_{n,m}}{\sigma^2} \right), \quad \forall m \in \mathcal{M}, \forall n \in \mathcal{N}, \quad (3)$$

respectively, where σ^2 denotes the white Gaussian noise power, B denotes the channel bandwidth, P_m denotes the transmit power of vessel m , P_n denotes the transmit power of n -th USV cluster header.

Vessels can offload their tasks to USV clusters or the cloud center. We introduce a binary variable $\alpha_{m,n} \in \{0, 1\}$, $\forall m \in \mathcal{M}, \forall n \in \mathcal{N} \cup \{0\}$ to represent computation offloading decision of vessel m . Specifically, $\alpha_{m,n} = 1$ indicates that vessel m offloads the task to a USV cluster $n \in \mathcal{N}$ or the cloud center $n = \{0\}$, and $\alpha_{m,n} = 0$ otherwise. Each vessel can offload its computation tasks to either one USV cluster for edge computing or the cloud center for cloud computing, satisfying

$$\sum_{n \in \mathcal{N} \cup \{0\}} \alpha_{m,n} = 1, \quad \forall m \in \mathcal{M}. \quad (4)$$

Case 1. Edge Computing: In the case that the computation tasks are offloaded to the edge server, the response time of the vessel is the sum of the uplink transmission time from vessel m to a USV cluster n , denoted as $T_{m,n}^{\text{ul}}$, the computation time at the USV cluster n , denoted as $T_{m,n}^{\text{comp}}$, and the downlink transmission time from a USV cluster n to vessel m , denoted as $T_{n,m}^{\text{dl}}$. Thus, we have

$$T_{m,n} = T_{m,n}^{\text{ul}} + T_{m,n}^{\text{comp}} + T_{n,m}^{\text{dl}}, \quad \forall m \in \mathcal{M}, \forall n \in \mathcal{N}. \quad (5)$$

The uplink transmission time $T_{m,n}^{\text{ul}}$ is given by

$$T_{m,n}^{\text{ul}} = \alpha_{m,n} \frac{I_m b_m^{\text{in}}}{R_{m,n}}, \quad \forall m \in \mathcal{M}, \forall n \in \mathcal{N}, \quad (6)$$

where b_m^{in} denotes the average input data volume of the computation task of vessel m .

The computation resource of USV cluster n is $C_n = \sum_{u \in \mathcal{Z}_n} C_u$, where C_u is the computation resource equipped in

USV u . The computation time at USV cluster n to process the offloaded tasks of vessel m is

$$T_{m,n}^{\text{comp}} = \alpha_{m,n} \frac{I_m c_m}{C_n}, \forall m \in \mathcal{M}, \forall n \in \mathcal{N}, \quad (7)$$

where c_m denotes the required CPU cycles for computing the computation task of vessel m .

The downlink transmission time of computation results to be sent from USV cluster n to vessel m is given by

$$T_{n,m}^{\text{dl}} = \alpha_{m,n} \frac{I_m b_m^{\text{out}}}{R_{n,m}}, \forall m \in \mathcal{M}, \forall n \in \mathcal{N}, \quad (8)$$

where b_m^{out} denotes average data volume of the task computation result of vessel m .

Note that the total edge computation resource to serve vessels cannot exceed the maximum computation resource of the USV cluster n ,

$$\sum_{m \in \mathcal{M}} \alpha_{m,n} \lambda_m c_m \leq C_n, \forall n \in \mathcal{N}. \quad (9)$$

Case 2. Cloud Computing: In the case that the computation tasks are offloaded to the cloud, the response time is the sum of the uplink transmission time from vessel m to the cloud center, denoted as $T_{m,0}^{\text{ul}}$, the computation time at the cloud center, denoted as $T_{m,0}^{\text{comp}}$, and the downlink transmission time from the cloud center to vessel m , denoted as $T_{0,m}^{\text{dl}}$. Thus, we have

$$T_{m,0} = T_{m,0}^{\text{ul}} + T_{m,0}^{\text{comp}} + T_{0,m}^{\text{dl}}, \forall m \in \mathcal{M}. \quad (10)$$

The uplink transmission time $T_{m,0}^{\text{ul}}$ is given by

$$T_{m,0}^{\text{ul}} = \alpha_{m,0} \left(\frac{I_m b_m^{\text{in}}}{R_{m,n}} + \frac{I_m b_m^{\text{in}}}{R_{n,0}} + \tau_0 \right), \forall m \in \mathcal{M}, \forall n \in \mathcal{N}, \quad (11)$$

where the first term in the braces represents the task transmission time from vessel m to the nearest USV cluster n , the second term is the transmission time from the USV cluster n to the coastal BS, and the third term τ_0 denotes the transmission time from the coastal BS to the cloud center.

Generally, the computation resource of the cloud center is much greater than that in the edge server, and we have $C_0 \gg C_n$. The computation time at the cloud center to process the offloaded tasks of vessel m is

$$T_{m,0}^{\text{comp}} = \alpha_{m,0} \frac{I_m c_m}{C_0}, \forall m \in \mathcal{M}. \quad (12)$$

The computation results are then transmitted from the cloud center to vessel m through the coastal BS, and the transmission time is thus given by

$$T_{0,m}^{\text{dl}} = \alpha_{m,0} \left(\tau_0 + \frac{I_m b_m^{\text{out}}}{R_{0,n}} + \frac{I_m b_m^{\text{out}}}{R_{n,m}} \right), \forall m \in \mathcal{M}, \forall n \in \mathcal{N}, \quad (13)$$

where the first term in the braces, τ_0 , denotes the transmission time of computation results from the cloud center to the coastal BS, the second term represents the transmission time from the coastal BS to the USV cluster n , and the third term represents the transmission time from the USV cluster n to vessel m .

B. Problem Formulation

The objective is to minimize the expected response time by jointly optimizing the deployment of USV clusters $\mathbf{L}^{\text{usv}} = \{\mathbf{L}_n, \forall n \in \mathcal{N}\}$ and the computation offloading decisions for

vessels $\alpha = \{\alpha_{m,n}, \forall m \in \mathcal{M}, \forall n \in \mathcal{N} \cup \{0\}\}$, while satisfying the edge computing resource constraint. Mathematically, we formulate the Expected Response Time Minimization (ERTM) problem as follows,

(ERTM) :

$$\min_{\mathbf{L}^{\text{usv}}, \alpha} T = \frac{1}{M} \sum_{m \in \mathcal{M}} (T_{m,n} + T_{m,0}) \quad (14)$$

$$\text{s.t. } \alpha_{m,n} \in \{0, 1\}, \forall m \in \mathcal{M}, \forall n \in \mathcal{N} \cup \{0\}, \quad (14a)$$

$$\sum_{n \in \mathcal{N} \cup \{0\}} \alpha_{m,n} = 1, \forall m \in \mathcal{M}, \quad (14b)$$

$$\sum_{m \in \mathcal{M}} \alpha_{m,n} \lambda_m c_m \leq C_n, \forall n \in \mathcal{N}, \quad (14c)$$

$$0 \leq x_n \leq x_{\max}, 0 \leq y_n \leq y_{\max}, \forall n \in \mathcal{N}. \quad (14d)$$

Constraints (14a) and (14b) represent that each vessel can offload its tasks to either one USV cluster or the cloud center. Constraint (14c) means that the total computation resource cannot exceed the maximum computation resource of the USV cluster. Constraint (14d) is the deployment area constraint.

IV. DUAL-LAYER REINFORCEMENT LEARNING ALGORITHM DESIGN

In this section, we first reformulate the ERTM problem as an MDP, and then propose a dual-layer reinforcement learning framework to handle the hybrid action spaces and to attain the optimal solution.

A. MDP Formulation

In this subsection, we reformulate the ERTM problem as an MDP. For the DLRL algorithm, the coastal BS is modeled as an agent, and the MEC-assisted maritime wireless network is the environment. The state space, the action space, and the reward function of the MDP are defined as follows.

1) *Action Space:* The action space in the t -th round is $\mathbf{a}_t = \{\mathbf{v}_t, \boldsymbol{\vartheta}_t, \mathbf{D}_t\}$, where $\mathbf{v}_t = \{v_{m,t} | m \in \mathcal{M}\}$ denotes the computation offloading decisions of vessels, $\boldsymbol{\vartheta}_t = \{\vartheta_{n,t} | \vartheta_{n,t} \in [0, 2\pi), \forall n \in \mathcal{N}\}$ denotes the search angle for USV clusters, and $\mathbf{D}_t = \{D_{n,t} | D_{n,t} \in [0, D_{\max}], \forall n \in \mathcal{N}\}$ denotes the search distance of USV clusters within a maximal search distance $D_{\max}$.

2) *State Space:* The state space in the t -th round is $\mathbf{s}_t = \{\mathbf{L}_t^{\text{usv}}, \mathbf{h}_t\}$, where $\mathbf{L}_t^{\text{usv}} = \{\mathbf{L}_{n,t} | n \in \mathcal{N}\}$ denotes the locations of USV clusters, and $\mathbf{h}_t = \{h_{m,n,t} | m \in \mathcal{M}, n \in \mathcal{N}\}$ denotes the channel gain between vessels and USV clusters.

3) *Reward Function:* According to the problem formulation in Section III, the objective of the ERTM problem is to minimize the expected response time by optimizing the deployment of each USV cluster and computation offloading decisions. Therefore, the reward function of the DLRL algorithm should be inversely proportional to the expected response time, which is defined as

$$r_t = \frac{1}{T(\mathbf{s}_t, \mathbf{a}_t)}, \quad (15)$$

where $T(\mathbf{s}_t, \mathbf{a}_t)$ is the expected response time of all vessels at the state $\mathbf{s}_t$ and taking the action $\mathbf{a}_t$.

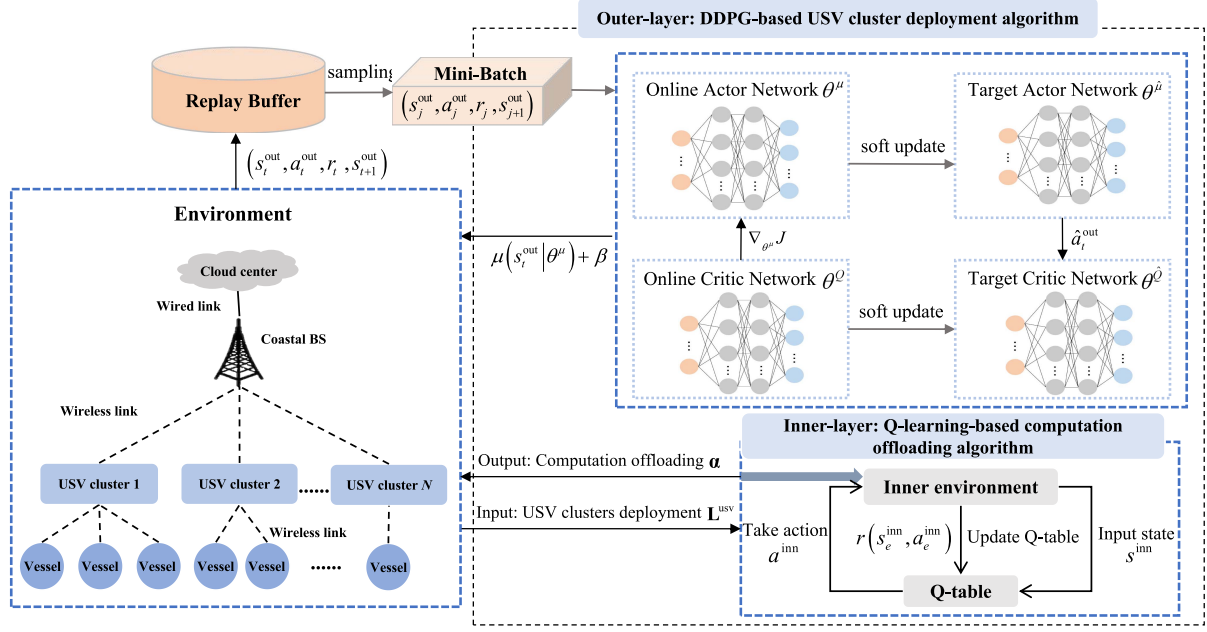


Fig. 2. Framework of the DLRL algorithm.

B. Dual-Layer Reinforcement Learning

We propose a dual-layer learning framework to solve the formulated problem. As shown in Fig. 2, the outer layer DDPG is employed for the USV deployment in continuous space and the inner layer Q-learning is employed for the computation offloading decisions in discrete space. Specifically, the inner layer learning is executed at the beginning of each epoch, which can obtain the best computation offloading results and interact with the environment. The results of the inner layer learning is used to further train the outer layer Deep Neural Networks (DNNs). In the outer layer, we utilize a DDPG to train the USV deployment design, where two main DNNs, i.e., the actor network and the critic network, are trained with the same structure but different parameters. In the DLRL, a mini-batch is used to sample the historical experience stored in the replay buffer for updating network parameters. Finally, the USV deployment actions and computation offloading selection actions are coordinated to achieve convergence. The detailed designs of the outer layer and inner layer learning algorithms are presented as follows.

1) *Outer Layer Framework of DLRL Algorithm:* A deep reinforcement learning algorithm based on DDPG is proposed to optimize the deployment locations of USVs in the continuous action space. Based on the MDP, DDPG can guide the agent to efficiently approach the near-optimal solution with continuous variables by exploring and interacting with the environment. The agent observes the current environment state, selects an action, receives an immediate reward based on the current state and action, and then observes the next state. Specifically, for the outer layer, the action space in the t -th round is $\mathbf{a}_t^{\text{out}} = \{\mathbf{p}_t, \mathbf{D}_t\}$, and the state space in the t -th round is $\mathbf{s}_t^{\text{out}} = \{\mathbf{L}_t^{\text{usv}}, \mathbf{h}_t\}$.

The DDPG algorithm consists of two main networks, i.e., the actor network and the critic network. Both the actor and critic networks feature two subnets, i.e., the online network and the target network with the same structure. The critic network

is trained to approximate the Q-values based on the state and action. The actor network is trained to generate a deterministic policy and returns the corresponding action based on the current state. Besides, experience replay buffer and target networks are employed to enhance the stability and convergence of the learning process in the online networks.

The goal of DDPG is to find optimal policies that maximize the cumulative reward value. A policy μ can be considered as a function that maps states to actions. Thus, we can obtain the action-value function as follows,

$$Q(\mathbf{s}_t^{\text{out}}, \mathbf{a}_t^{\text{out}} | \theta^Q) = \mathbb{E}[r_t + \zeta^{\text{out}} Q(\mathbf{s}_{t+1}^{\text{out}}, \mathbf{a}_{t+1}^{\text{out}} | \theta^Q)], \quad (16)$$

where θ^Q is the weight of the online critic network, $\zeta^{\text{out}} \in [0, 1]$ is the discount factor that represents the uncertainty of future revenue, $\mathbb{E}[\cdot]$ is the expectation.

The training process can be described as follows. Initially, the actor network adds the noise β and generates the action $\mathbf{a}_t^{\text{out}} = \mu(\mathbf{s}_t^{\text{out}} | \theta^\mu) + \beta$, where θ^μ is the weight of the online actor network. Upon interaction with the environment using action $\mathbf{a}_t$, the agent receives a reward r_t and transitions to the next state $\mathbf{s}_{t+1}^{\text{out}}$. This experience $(\mathbf{s}_t^{\text{out}}, \mathbf{a}_t^{\text{out}}, r_t, \mathbf{s}_{t+1}^{\text{out}})$ is stored in the experience replay buffer $\mathbf{O}$. After that, a set of historical experiences from the replay buffer are randomly sampled into a mini-batch buffer, such as $(\mathbf{s}_j^{\text{out}}, \mathbf{a}_j^{\text{out}}, r_j, \mathbf{s}_{j+1}^{\text{out}})$, to update the network weights.

The critic network updates its online network by minimizing loss of the critic network, which is approximated by

$$L(\theta^Q) = \mathbb{E}[(Q(\mathbf{s}_j^{\text{out}}, \mathbf{a}_j^{\text{out}} | \theta^Q) - Y_j)^2], \quad (17)$$

where $Y_j = r_j + \zeta Q(\mathbf{s}_{j+1}^{\text{out}}, \hat{\mu}(\mathbf{s}_{j+1}^{\text{out}} | \theta^{\hat{\mu}}) | \theta^{\hat{Q}})$ is the Q-value produced by the target critic network, $\theta^{\hat{\mu}}$ and $\theta^{\hat{Q}}$ are weights of the target actor network and the target critic network, respectively. The symbol $\hat{\mu}(\cdot | \theta^{\hat{\mu}})$ means the action policy obtained by the target actor network. Correspondingly, the weight θ^Q of the

online critic network is updated as

$$\nabla_{\theta^Q} L(\theta^Q) = \mathbb{E}[(Q(\mathbf{s}_j^{\text{out}}, \mathbf{a}_j^{\text{out}} | \theta^Q) - Y_j) \times \nabla_{\theta^Q} Q(\mathbf{s}_j^{\text{out}}, \mathbf{a}_j^{\text{out}} | \theta^Q)], \quad (18)$$

$$\theta^Q = \theta^Q + \gamma_Q \nabla_{\theta^Q} L(\theta^Q), \quad (19)$$

where ∇_{θ^Q} denotes the gradient vector with the weight θ^Q , γ_Q is the learning rate of the online critic network.

With the Adam optimizer, the gradient of the weight θ^μ is derived as $\nabla_{\theta^\mu} \mu(\mathbf{s}_j^{\text{out}} | \theta^\mu)$. Based on the sampling mini-batch, the policy gradient of the online actor network with the weight θ^μ is updated as

$$\nabla_{\theta^\mu} J(\theta^\mu) = \mathbb{E}[\nabla_a Q(\mathbf{s}_j^{\text{out}}, a | \theta^Q) |_{a=\mu(\mathbf{s}_j^{\text{out}} | \theta^\mu)} \nabla_{\theta^\mu} \mu(\mathbf{s}_j^{\text{out}} | \theta^\mu)]. \quad (20)$$

Correspondingly, the weight θ^μ of the online actor-network is expressed as

$$\theta^\mu = \theta^\mu + \gamma_\mu \nabla_{\theta^\mu} J(\theta^\mu), \quad (21)$$

where γ_μ is the learning rate of the online actor network.

With the soft update strategy, the weights of the target actor network and the target critic network are updated as

$$\theta^{\hat{\mu}} \leftarrow \delta \theta^\mu + (1 - \delta) \theta^{\hat{\mu}}, \quad (22)$$

$$\theta^{\hat{Q}} \leftarrow \delta \theta^Q + (1 - \delta) \theta^{\hat{Q}}, \quad (23)$$

respectively. Here, $\delta \in [0, 1]$ is the soft update coefficient.

2) *Inner Layer Framework of DLRL Algorithm:* Q-learning is an effective model-free reinforcement learning algorithm based on value iteration, which can find the optimal action in a finite MDP [37]. In the inner layer, the action space is $\mathbf{a}_e^{\text{inn}} = \{\mathbf{v}_e\}$, and the state space is $\mathbf{s}_e^{\text{inn}} = \{\mathbf{h}_e\}$. The agent computes the value of each state-action pair, and updates the Q-value in a Q-table after each interaction.

The ϵ -greedy policy is employed to strike a balance between exploitation and exploration. Specifically, the agent selects the optimal action based on the current Q-table with a probability of ϵ for exploitation. With a probability of $1 - \epsilon$, the agent selects a random action for exploration, which enables the agent to try out new actions and potentially discover better ones that may not have been selected through exploitation. Thus, the action with ϵ -greedy policy in the e -th epoch is given by

$$\mathbf{a}_e^{\text{inn}} = \begin{cases} \arg\max_{\mathbf{a}_e^{\text{inn}} \in \mathbf{a}^{\text{inn}}} Q(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}}), & \text{with probability } \epsilon, \\ \text{a random action,} & \text{with probability } 1 - \epsilon. \end{cases} \quad (24)$$

Q-value is a state-action value, which describes the benefit of an agent performing a particular action in a state. Therefore, the ultimate goal of Q-learning is to appropriately update the Q-table such that the agent obtains the maximum reward. The Q-value in the Q-table can be calculated by

$$Q(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}}) \leftarrow Q(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}}) + \gamma^{\text{inn}} [r(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}}) + \zeta^{\text{inn}} \max_{\mathbf{a}_{e+1}^{\text{inn}}} Q(\mathbf{s}_{e+1}^{\text{inn}}, \mathbf{a}_{e+1}^{\text{inn}}) - Q(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}})], \quad (25)$$

where $Q(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}})$ is the state-action value, $\zeta^{\text{inn}} \in \{0, 1\}$ is the discount factor representing the weight of future rewards, and γ^{inn} is the learning rate.

The proposed DLRL algorithm, including the inner layer Q-learning and outer layer DDPG is presented in Algorithm 1.

Algorithm 1: DLRL Algorithm.

Input: Episode length E^{out} and E^{inn} , training round T , experience replay buffer $\mathbf{O}$, mini-batch size o , learning rate for actor network γ_μ , learning rate for critic network γ_Q , discount factor ζ^{out} , learning rate of Q-learning γ^{inn} , soft update coefficient δ ;

Output: Locations of USV clusters $\mathbf{L}^{\text{usv}}$, computation offloading decisions α ;

- 1: Initialize online actor network and online critic network with the weights θ^μ and θ^Q ;
- 2: Initialize target actor network and target critic network with the weights $\theta^{\hat{\mu}}$ and $\theta^{\hat{Q}}$;
- 3: Initialize the locations of USV clusters $\mathbf{L}_0^{\text{usv}}$;
- 4: Empty experience replay buffer;
- 5: **for** $episode = 1$ to E^{out} **do**
- 6: Inner layer Q-learning
- 7: Initialize $Q(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}}) = 0$;
- 8: **for** $e = 1$ to E^{inn} **do**
- 9: Select an action $\mathbf{a}_e^{\text{inn}}$ according to the ϵ -greedy policy at state $\mathbf{s}_e^{\text{inn}}$;
- 10: **if** $\text{rand}(\cdot) < \epsilon$ **then**
- 11: Choose the action $\mathbf{a}_e^{\text{inn}}$ that maximizes the Q-value $Q(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}})$;
- 12: **else**
- 13: Randomly select an action $\mathbf{a}_e^{\text{inn}}$;
- 14: **end if**
- 15: Execute action $\mathbf{a}_e^{\text{inn}}$, and calculate the reward value $r(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}})$;
- 16: Update $Q(\mathbf{s}_e^{\text{inn}}, \mathbf{a}_e^{\text{inn}})$ by (25);
- 17: **end for**
- 18: Obtain the computation offloading decisions α based on the result of the previous episode;
- 19: Outer layer DDPG
- 20: Obtain the initial state $\mathbf{s}_1^{\text{out}}$;
- 21: **for** training round $t = 1$ to T **do**
- 22: Update the state $\mathbf{s}_t^{\text{out}}$;
- 23: Select the action $\mathbf{a}_t^{\text{out}} = \mu(\mathbf{s}_t^{\text{out}} | \theta^\mu) + \beta$ by the online actor network;
- 24: Obtain the next state $\mathbf{s}_{t+1}^{\text{out}}$ and the reward r_t based on the state $\mathbf{s}_t^{\text{out}}$ and action $\mathbf{a}_t^{\text{out}}$;
- 25: Store the experience $(\mathbf{s}_t^{\text{out}}, \mathbf{a}_t^{\text{out}}, r_t, \mathbf{s}_{t+1}^{\text{out}})$ in $\mathbf{O}$;
- 26: Random sample a mini-batch of o experiences $(\mathbf{s}_j^{\text{out}}, \mathbf{a}_j^{\text{out}}, r_j, \mathbf{s}_{j+1}^{\text{out}})$ from $\mathbf{O}$;
- 27: Update the critic network with (18) and (19);
- 28: Update the actor network with (20) and (21);
- 29: Update the weights $\theta^{\hat{\mu}}$ and $\theta^{\hat{Q}}$ by (22) and (23);
- 30: **end for**
- 31: **end for**
- 32: Return $\mathbf{L}^{\text{usv}}, \alpha$.

C. Computational Complexity Analysis

In this subsection, we discuss the computational complexity of the proposed DLRL algorithm. For the inner layer, the computational complexity of the Q-learning depends upon the state space size and action space size. Thus, the computational complexity is $\mathcal{O}(E^{\text{inn}} |\mathbf{s}^{\text{inn}}| |\mathbf{a}^{\text{inn}}|)$, where E^{inn} is the total number of episodes in the training process, $|\mathbf{s}^{\text{inn}}|$ is the state space size, and $|\mathbf{a}^{\text{inn}}|$

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Deployment area	10nmile × 10nmile
The number of vessels, M	20~45
The number of USVs, U	10~36
The number of tasks per vessel, I_m	5~10
The computation resource of USV u , C_u	3GHz
The computation resource of the cloud center, C_0	100GHz
Channel bandwidth, B	1MHz
Noise power spectral density, σ^2	-114dBm
The carrier wavelength, l	0.05m
The antenna height of USV u , H_u	5m
The antenna height of vessel m , H_m	9.8m
The transmit power of USV u , P_u	44.7dBm
The transmit power of vessel m , P_m	47dBm

is the action space size. For the outer layer, the computational complexity of the DDPG usually depends on the actor and critic networks. The actor network has L_a layers with F_i neurons in the i -th layer ($i \leq L_a$). The complexity of the i -th layer is $\mathcal{O}(F_{i-1}F_i + F_iF_{i+1})$. The complexity of the actor network is $\mathcal{O}(\sum_{i=2}^{L_a-1}(F_{i-1}F_i + F_iF_{i+1}))$ [38]. The critic network has L_c layers with G_j in the j -th layer ($j \leq L_c$). The complexity of the j -th layer is $\mathcal{O}(G_{j-1}G_j + G_jG_{j+1})$. The complexity of the critic network is $\mathcal{O}(\sum_{j=2}^{L_c-1}(G_{j-1}G_j + G_jG_{j+1}))$. Thus, the complexity of the outer layer algorithm is denoted as $\mathcal{O}(E^{\text{out}}T(\sum_{i=2}^{L_a-1}(F_{i-1}F_i + F_iF_{i+1}) + \sum_{j=2}^{L_c-1}(G_{j-1}G_j + G_jG_{j+1})))$. Thus, the overall complexity of the DLRL algorithm is $\mathcal{O}(E^{\text{out}}T(\sum_{i=2}^{L_a-1}(F_{i-1}F_i + F_iF_{i+1}) + \sum_{j=2}^{L_c-1}(G_{j-1}G_j + G_jG_{j+1})) + E^{\text{out}}E^{\text{inn}}|s^{\text{inn}}||a^{\text{inn}}|)$.

V. PERFORMANCE EVALUATION

We develop a simulator in Python to evaluate the performance of the proposed DLRL framework for an MEC-assisted maritime wireless network with multiple USVs as described in Section III. The DLRL algorithm is implemented in TensorFlow. we set $\gamma_\mu = 0.0001$ and $\gamma_Q = 0.0001$ as the learning rate for the actor network and the critic network, respectively, and $\gamma^{\text{inn}} = 0.01$ as the learning rate for the inner layer. Both the actor network and the critic network have three hidden layers with (256,128,64) neurons. The main simulation parameters used in the experiments are listed in Table II, as in [39], [40], [41].

We also implement four benchmark algorithms for performance comparison as follows.

- 1) Particle Swarm Optimization (PSO) and Greedy (PSO-G) algorithm [42]. We select PSO algorithm as the first benchmark algorithm for performance comparison as it provides a generic solution to handle complex optimization problems with mixed variables. Specifically, a PSO algorithm based on swarm intelligence search is adopted to optimize the locations of USV clusters; and a greedy algorithm is used to optimize the computation offloading decisions. The USV cluster deployment and computation offloading decisions are updated iteratively until a convergence is reached.

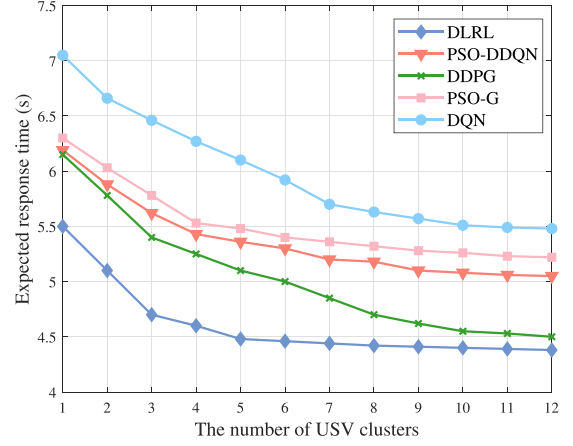


Fig. 3. Performance comparison of the expected response time ($M = 40$ vessels and $U = 24$ USVs).

- 2) PSO and Double Deep Q-Network (PSO-DDQN) algorithm [43]. We selected the PSO-DDQN algorithm as the second benchmark due to its effectiveness in addressing MINLP problems. Specifically, the PSO algorithm is employed to optimize the locations of USV clusters, while the DDQN algorithm is utilized to refine the computation offloading decisions. This combined approach optimizes the computation offloading strategies based on the USVs' locations, iteratively updating until convergence is achieved.
- 3) Deep Q Network (DQN) algorithm [44]. A deep reinforcement learning algorithm based on Deep Q-Network (DQN) is selected as the third benchmark algorithm due to its high performance in tackling problems with discrete action space. In order to handle mixed action spaces, the actions of USV deployment are discretized, while the action of offloading is set as that in Section IV-A.
- 4) DDPG algorithm. A deep reinforcement learning algorithm based on DDPG is selected as the fourth benchmark algorithm due to its high performance in tackling problems with continuous action space [45]. To handle the mixed action spaces, the discrete offloading action is approximated in continuous action space, while the continuous actions of USV deployment are set as that in Section IV-A.

Fig. 3 compares the performance of the proposed algorithm with that of the four benchmark algorithms versus different numbers of USV clusters. The number of vessels and USVs are set to $M = 40$ and $U = 24$. The computation resource of each USV is 3 GHz. Each vessel carries out a variable number of computation tasks from 5 to 10. It is shown that the proposed learning algorithm outperforms the PSO-G, PSO-DDQN, DQN, and DDPG in terms of the achieved response time. The DDPG uses continuous actions to approximate the discrete offloading decisions, while the DQN algorithm discretizes the actions of USV deployment, resulting in reduced accuracy, compared with the proposed dual-layer learning solution. The PSO-G and PSO-DDQN algorithms involve iterative optimization for both discrete and continuous variables. Generally, PSO-G prioritizes exploration to find the optimal solutions while DDPG and DQN algorithms prioritize exploitation by leveraging learned policies to maximize the rewards. PSO-DDQN algorithm as a form of

deep reinforcement learning by incorporating PSO and DDQN for exploration and exploitation. The proposed DLRL algorithm that integrates DDPG and Q-learning to handle the continuous and discrete actions achieves the best performance. It can also be seen that the expected response time decreases when the number of USV clusters increases from 1 to 5, and the decrease rate becomes much slower for $N > 5$. As the number of USV clusters increases, the distance between the vessel and the USV cluster decreases, which effectively decreases the transmission time. The results suggest that it may not be necessary to deploy each individual USV to the optimal location, and an appropriate number of USV clusters can be determined to facilitate the efficient deployment without notably reducing the network performance.

Fig. 4 studies the performance of the proposed algorithm under different numbers of tasks. The edge computing ratio, defined as the number of computation tasks offloaded to the USVs for edge computing over the total number of tasks, is shown in Fig. 4(a). For $U = 24$ and $N = 5$, when the number of vessels exceeds $M > 25$, some tasks cannot be offloaded to the edge server but have to be forwarded to the remote cloud for data processing, due to the limited computation resources of deployed USVs. Thus, the edge computing ratio decreases with the number of vessels, and the corresponding expected response time increases as shown in Fig. 4(b). We further increase the number of USVs to $U = 36$, and more computation tasks can be offloaded to the USVs with more computation resources, and the edge computing ratio is higher than that with $U = 24$, and a smaller expected response time is achieved correspondingly, as shown in Fig. 4(c).

Fig. 5 studies the impact of the computation resources on the performance of the proposed algorithm. The more computation resources, the more tasks can be offloaded to the edge server to achieve a lower response time. As shown in Fig. 5, when the computation resource is larger than 100, i.e., in the case of $U > 20$ and $C_u = 5$ GHz, or $U > 28$ and $C_u = 4$ GHz, all computation tasks of vessels can be offloaded to the edge servers to attain the minimum expected response time.

The convergence performance of the proposed DLRL algorithm under different numbers of USV clusters and vessels is shown in Fig. 6. In Fig. 6(a), the number of vessels and USVs are set to $M = 15$ and $U = 10$, respectively. It is found that the achieved expected response time for $N = 10$ is reduced compared with the cases of $N = 1, 3, 5$ clusters. It is also observed that the convergence time increases with N due to a larger searching space of the optimal locations. For example, when the number of USV clusters is $N = 10$, the convergence performance increases by about 1100, 800, 600 episodes compared with the cases of $N = 1, 3, 5$, respectively. Similar results can be observed in Fig. 6(b) where the number of vessels and USVs increases to $M = 40$ and $U = 24$, respectively. Compared with the case of $N = 5$, the expected response time for $N = 10$ is reduced by 1.8% at the cost of increased convergence time. In a large scale IoV networks with a high number of vessels and USVs, clustering provides an efficient and scalable solution to strike a balance between the response time and the convergence time.

Fig. 7 compares the convergence performance of the proposed four learning algorithms, i.e., DLRL, PSO-DDQN, DDPG, and

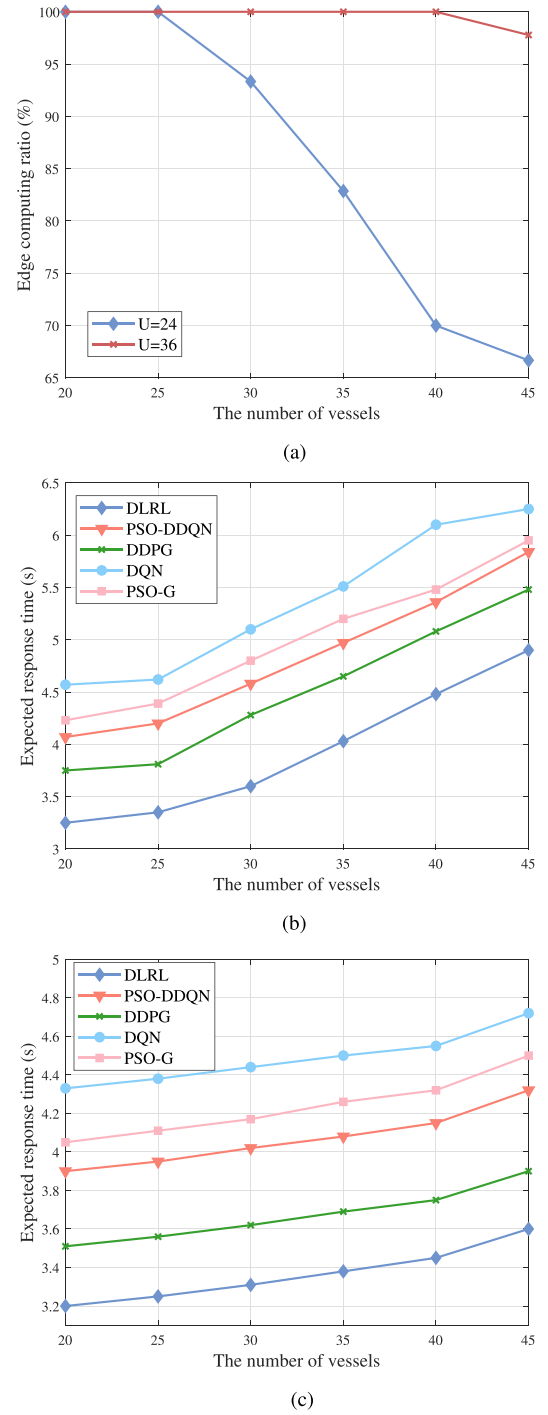
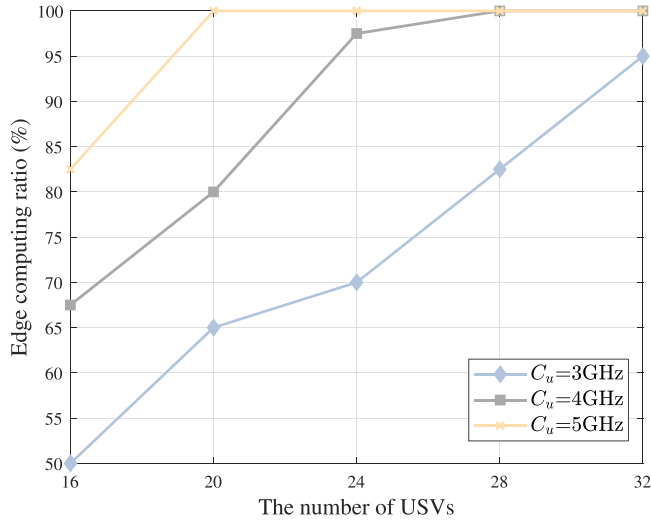


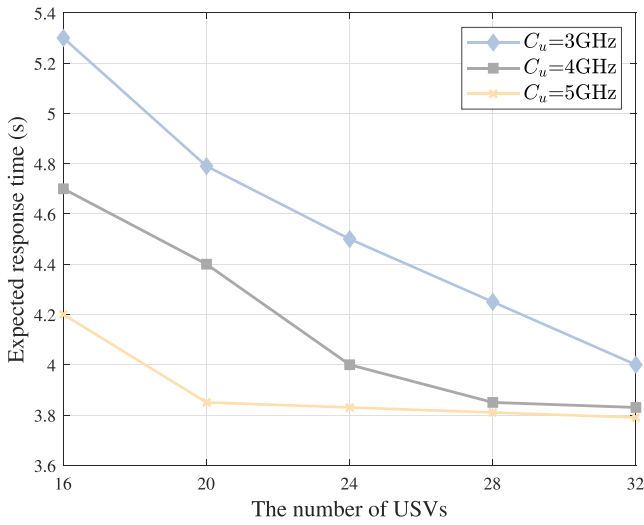
Fig. 4. Performance study of the edge computing ratio and expected response time ($N = 5$ USV clusters). (a) Edge computing ratio. (b) Expected response time ($U = 24$ USVs). (c) Expected response time ($U = 36$ USVs).

DQN.² The proposed DLRL algorithm typically converges in about 400 episodes, while the DDPG algorithm requires approximately 800 episodes to converge. In addition, the final accumulated reward of the DLRL algorithm is slightly higher

²The converge performance of PSO-G is not presented as PSO-G is not typically based on rewards as the reinforcement learning algorithms.



(a)



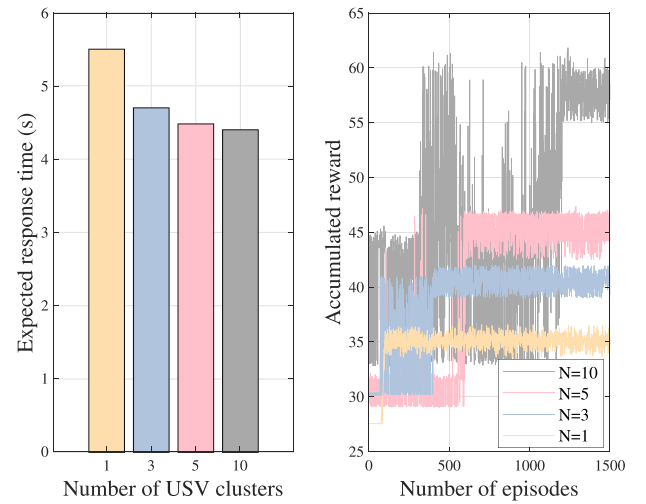
(b)

 Fig. 5. The impact of the computation resources ($M = 40$ vessels and $N = 5$ USV clusters). (a) Edge computing ratio. (b) Expected response time.

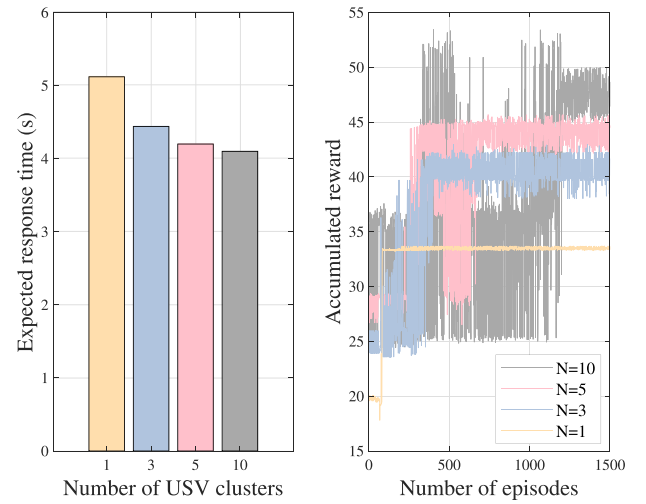
than that of the DDPG algorithm because the DDPG approximates the discrete offloading action in continuous action space, resulting in reduced accuracy. Although the DQN algorithm achieves a good convergence performance, the accumulated reward learned from the DQN is lower than that of the DLRL, as DQN discretizes the continuous action space which results in performance degradation. For the PSO-DDQN algorithm, the convergence performance is inferior to that of the DLRL, as the incorporation of PSO increases the complexity of exploration, which ultimately degrades overall performance. The proposed DLRL algorithm achieves the highest rewards with good convergence performance compared to PSO-DDQN, DDPG and DQN primarily due to its effective handling of hybrid actions.

VI. CONCLUSION

In this paper, we have studied the mobile edge deployment and resource management for MEC-assisted maritime wireless



(a)



(b)

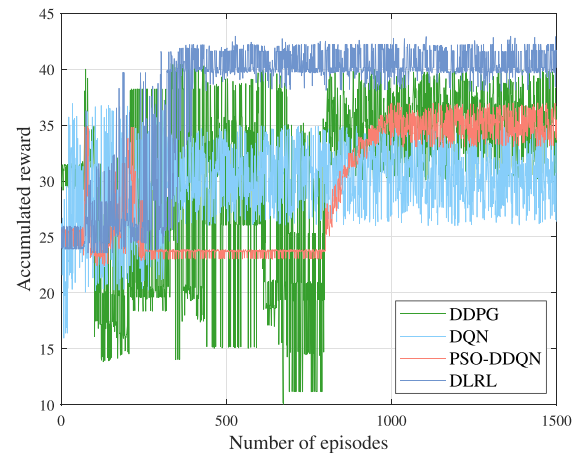
 Fig. 6. Convergence performance versus different numbers of USV clusters and vessels. (a) $M = 15$ vessels, $U = 10$ USVs. (b) $M = 24$ vessels, $U = 40$ USVs.


Fig. 7. Convergence performance of DLRL, PSO-DDQN, DDPG and DQN algorithms.

networks, where USVs with computation resources are deployed to provide edge computation services for vessels. A dual-layer reinforcement learning framework has been proposed to optimize the deployment of USVs and computation offloading decisions. Simulation results have demonstrated that the USV cluster deployment provides an efficient and scalable solution for large scale maritime wireless networks with a high number of USVs and vessels. Compared with four benchmark algorithms in the literature, the proposed DLRL algorithm has achieved a significant improvement in reducing the expected response time. In the future, we will further study the computation offloading in a heterogeneous MEC-assisted maritime wireless network with different types of tasks with different QoS requirements.

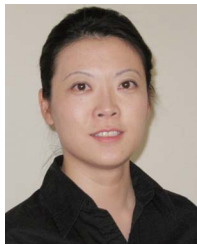
REFERENCES

- [1] L. Lyu, Z. Chu, B. Lin, Y. Dai, and N. Cheng, "Fast trajectory planning for UAV-enabled maritime IoT systems: A fermat-point based approach," *IEEE Wireless Commun. Lett.*, vol. 11, no. 2, pp. 328–332, Feb. 2022.
- [2] B. Lin, J. Duan, M. Han, and L. X. Cai, *Next Generation Marine Wireless Communication Networks*. Cham, Switzerland: Springer, 2022.
- [3] T. Wei, W. Feng, Y. Chen, C.-X. Wang, N. Ge, and J. Lu, "Hybrid satellite-terrestrial communication networks for the maritime Internet of Things: Key technologies, opportunities, and challenges," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8910–8934, Nov. 2022.
- [4] H. Song, B. Gu, K. Son, and W. Choi, "Joint optimization of edge computing server deployment and user offloading associations in wireless edge network via a genetic algorithm," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2535–2548, Apr. 2023.
- [5] X. Zhang, Z. Li, C. Lai, and J. Zhang, "Joint edge server placement and service placement in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11261–11274, Jul. 2022.
- [6] W. Fan, L. Yao, J. Han, F. Wu, and Y. Liu, "Game-based multitype task offloading among mobile-edge-computing-enabled base stations," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17691–17704, Dec. 2022.
- [7] K. Cao, L. Li, Y. Cui, T. Wei, and S. Hu, "Exploring placement of heterogeneous edge servers for response time minimization in mobile edge-cloud computing," *IEEE Trans. Ind. Inform.*, vol. 17, no. 1, pp. 494–503, Jan. 2022.
- [8] Z. Tang, K. Yu, G. Yang, L. X. Cai, and H. Zhou, "New bridge to cloud: An ultra-dense leo assisted green computation offloading approach," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 552–564, Feb. 2024.
- [9] B. Cao et al., "Large-scale many-objective deployment optimization of edge servers," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3841–3849, Jun. 2021.
- [10] Y. Wu, L. P. Qian, K. Ni, C. Zhang, and X. Shen, "Delay-minimization nonorthogonal multiple access enabled multi-user mobile edge computation offloading," *IEEE J. Sel. Topics Signal Process.*, vol. 13, no. 3, pp. 392–407, Mar. 2019.
- [11] Y. Luo, W. Ding, and B. Zhang, "Optimization of task scheduling and dynamic service strategy for multi-UAV-enabled mobile-edge computing system," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 3, pp. 970–984, Mar. 2021.
- [12] H. Zhou, K. Jiang, X. Liu, X. Li, and V. C. M. Leung, "Deep reinforcement learning for energy-efficient computation offloading in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1517–1530, Jan. 2022.
- [13] J.-B. Wang, C. Zeng, C. Ding, H. Zhang, M. Lin, and J. Wang, "Unmanned surface vessel assisted maritime wireless communication toward 6 G: Opportunities and challenges," *IEEE Wireless Commun.*, vol. 29, no. 6, pp. 72–79, Jun. 2022.
- [14] C. Tang, H.-T. Zhang, and J. Wang, "Flexible formation tracking control of multiple unmanned surface vessels for navigating through narrow channels with unknown curvatures," *IEEE Trans. Ind. Electron.*, vol. 70, no. 3, pp. 2927–2938, Mar. 2023.
- [15] Z. Peng, D. Wang, Z. Chen, X. Hu, and W. Lan, "Adaptive dynamic surface control for formations of autonomous surface vehicles with uncertain dynamics," *IEEE Trans. Control Syst. Technol.*, vol. 21, no. 2, pp. 513–520, Feb. 2013.
- [16] T. Zhang, J. Lei, Y. Liu, C. Feng, and A. Nallanathan, "Trajectory optimization for UAV emergency communication with limited user equipment energy: A safe-DQN approach," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 3, pp. 1236–1247, Mar. 2022.
- [17] Y.-J. Chen and D.-Y. Huang, "Joint trajectory design and BS association for cellular-connected UAV: An imitation-augmented deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2843–2858, Apr. 2023.
- [18] L. Wang, H. Zhang, S. Guo, and D. Yuan, "Deployment and association of multiple UAVs in UAV-assisted cellular networks with the knowledge of statistical user position," *IEEE Trans. Wireless Commun.*, vol. 21, no. 8, pp. 6553–6567, Aug. 2022.
- [19] S. Zhang, W. Liu, and N. Ansari, "Completion time minimization for data collection in a UAV-enabled IoT network: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 11, pp. 14734–14742, Nov. 2023.
- [20] Z. Zhao, J. Shi, Z. Li, J. Si, P. Xiao, and R. Tafazolli, "Matching-aided-learning resource allocation for dynamic offloading in mmWave MEC system," *IEEE Trans. Wireless Commun.*, vol. 22, no. 11, pp. 7580–7591, Nov. 2023.
- [21] S. F. Abedin, A. Mahmood, N. H. Tran, Z. Han, and M. Gidlund, "Elastic O-RAN slicing for industrial monitoring and control: A distributed matching game and deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10808–10822, Oct. 2022.
- [22] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint computation offloading and user association in multi-task mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12313–12325, Dec. 2018.
- [23] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, and L. Li, "Delay-aware and energy-efficient computation offloading in mobile-edge computing using deep reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 3, pp. 881–892, Mar. 2021.
- [24] Y. Gao, J. Tao, H. Wang, Z. Wang, W. Sun, and C. Song, "Joint server deployment and task scheduling for the maximal profit in mobile-edge computing," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22501–22513, Dec. 2023.
- [25] T. K. Rodrigues, K. Suto, and N. Kato, "Edge cloud server deployment with transmission power control through machine learning for 6 G Internet of Things," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 4, pp. 2099–2108, Apr. 2021.
- [26] S. K. Kasi et al., "Heuristic edge server placement in industrial Internet of Things and cellular networks," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10308–10317, Jul. 2020.
- [27] J. Zhang, J. Lu, X. Yan, X. Xu, L. Qi, and W. Dou, "Quantified edge server placement with quantum encoding in internet of vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 9370–9379, Jul. 2022.
- [28] J. Zhang et al., "Stochastic computation offloading and trajectory scheduling for UAV-assisted mobile edge computing," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3688–3699, Feb. 2019.
- [29] H. Guo and J. Liu, "UAV-enhanced intelligent offloading for Internet of Things at the edge," *IEEE Trans. Ind. Inform.*, vol. 16, no. 4, pp. 2737–2746, Apr. 2020.
- [30] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC- and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.
- [31] P. Qin, Y. Fu, J. Zhang, S. Geng, J. Liu, and X. Zhao, "DRL-based resource allocation and trajectory planning for NOMA-enabled multi-UAV collaborative caching 6 G network," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 8750–8764, Jun. 2024.
- [32] Z. Han, T. Zhou, T. Xu, and H. Hu, "Joint user association and deployment optimization for delay-minimized UAV-aided MEC networks," *IEEE Wireless Commun. Lett.*, vol. 12, no. 10, pp. 1791–1795, Oct. 2023.
- [33] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning-based trajectory planning for multi-UAV assisted mobile edge computing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 1, pp. 73–84, Mar. 2021.
- [34] C. Zeng, J.-B. Wang, C. Ding, H. Zhang, M. Lin, and J. Cheng, "Joint optimization of trajectory and communication resource allocation for unmanned surface vehicle enabled maritime wireless networks," *IEEE Trans. Commun.*, vol. 69, no. 12, pp. 8100–8115, Dec. 2021.
- [35] C. Zeng, J.-B. Wang, C. Ding, M. Lin, and J. Wang, "MIMO unmanned surface vessels enabled maritime wireless network coexisting with satellite network: Beamforming and trajectory design," *IEEE Trans. Commun.*, vol. 71, no. 1, pp. 83–100, Jan. 2023.
- [36] M. Li, L. P. Qian, X. Dong, B. Lin, Y. Wu, and X. Yang, "Secure computation offloading for marine IoT: An energy-efficient design via cooperative jamming," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6518–6531, May 2023.
- [37] X. Wang, T. Jin, L. Hu, and Z. Qian, "Energy-efficient power allocation and q-learning-based relay selection for relay-aided D2D communication," *IEEE Trans. Veh. Technol.*, vol. 69, no. 6, pp. 6452–6462, Jun. 2020.

- [38] X. Yuan, S. Hu, W. Ni, R. P. Liu, and X. Wang, "Joint user, channel, modulation-coding selection, and RIS configuration for jamming resistance in multiuser OFDMA systems," *IEEE Trans. Commun.*, vol. 71, no. 3, pp. 1631–1645, Mar. 2023.
- [39] L. P. Qian, H. Zhang, Q. Wang, Y. Wu, and B. Lin, "Joint multi-domain resource allocation and trajectory optimization in UAV-assisted maritime IoT networks," *IEEE Internet Things J.*, vol. 10, no. 1, pp. 539–552, Jan. 2022.
- [40] N. Su, J.-B. Wang, C. Zeng, H. Zhang, M. Lin, and G. Y. Li, "Unmanned-surface-vehicle-aided maritime data collection using deep reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19773–19786, Oct. 2022.
- [41] N. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, Mar. 2019.
- [42] Z. Chen, H. Zheng, J. Zhang, X. Zheng, and C. Rong, "Joint computation offloading and deployment optimization in multi-UAV-enabled MEC systems," *Peer-to-Peer Netw. Appl.*, vol. 1, no. 15, pp. 194–205, 2022.
- [43] H. Sun, J. Wang, D. Yong, M. Qin, and N. Zhang, "Deep reinforcement learning-based computation offloading for mobile edge computing in 6 G," *IEEE Trans. Consum. Electron.*, vol. 70, no. 4, pp. 7482–7493, Nov. 2024.
- [44] V. Mnih et al., "Playing atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [45] T. Zhang, K. Zhu, and J. Wang, "Energy-efficient mode selection and resource allocation for D2D-enabled heterogeneous networks: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1175–1187, Feb. 2021.



Chaoyue Zhang (Member, IEEE) received the B.S. degree from Wuhan University of Science and Technology, Wuhan, China, in 2017, and the M.S. degree from Dalian Maritime University, Dalian, China, in 2020, where she is currently working toward the Ph.D. degree with the Information Science and Technology College. Her research interests include maritime communication networks, mobile edge computing, and resource management.



Bin Lin (Senior Member, IEEE) received the B.S. and M.S. degrees from Dalian Maritime University, Dalian, China, in 1999 and 2003, respectively and the Ph.D. degree from the Broadband Communications Research Group, Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON, Canada, in 2009. She is currently a Full Professor and the Dean of Communication Engineering with the College of Information Science and Technology, Dalian Maritime University. She has been a Visiting Scholar with George Washington University, Washington, DC, USA, from 2015 to 2016. Her research interests include wireless communications, network dimensioning and optimization, resource allocation, artificial intelligence, maritime communication networks, edge/cloud computing, wireless sensor networks, and Internet of Things. She is an Associate Editor of *IEEE TRANSACTION ON VEHICULAR TECHNOLOGY* and *IET Communications*.



Ziru Chen (Graduate Student Member, IEEE) received the B.E. degree in automation from Hunan University, Changsha, China, in 2014, and the M.S. degree in electric engineering from Illinois Institute of Technology, Chicago, IL, USA, in 2017, where he is currently working toward the Ph.D. degree with the Department of Electrical and Computer Engineering. His research interests include RF energy harvesting, stochastic geometry, nonorthogonal multiple access, and AI-assisted wireless networking.



Lin X. Cai (Senior Member, IEEE) received the M.A.Sc. and Ph.D. degrees in electrical and computer engineering from the University of Waterloo, Waterloo, ON, Canada, in 2005 and 2010, respectively. She is currently an Associate Professor with the Department of Electrical and Computer Engineering, Illinois Institute of Technology, Chicago, IL, USA. Her research interests include green communication and networking, intelligent radio resource management, and wireless Internet of Things. She was the recipient of the Best Paper Award from the IEEE Globecom 2011, an NSF Career Award in 2016, IIT Sigma Xi Research Award in the Junior Faculty Division in 2019, and the N2Women Stars award in Computer Networking and Communications in 2021. She is a Executive Editorial Committee Member of *IEEE TRANSACTION ON WIRELESS COMMUNICATIONS*, and an Associate Editor of *IEEE TRANSACTION ON VEHICULAR TECHNOLOGIES*, and a co-chair for IEEE conferences.



Jianli Duan received the M.S. and Ph.D. degrees from Dalian Maritime University, Dalian, China, in 2003 and 2020, respectively. She is currently an Associate Professor in School of Science, Qingdao University of Technology, Qingdao, China. Her research interests include maritime telecommunications and networking, wireless sensor networks, and network planning and optimization.